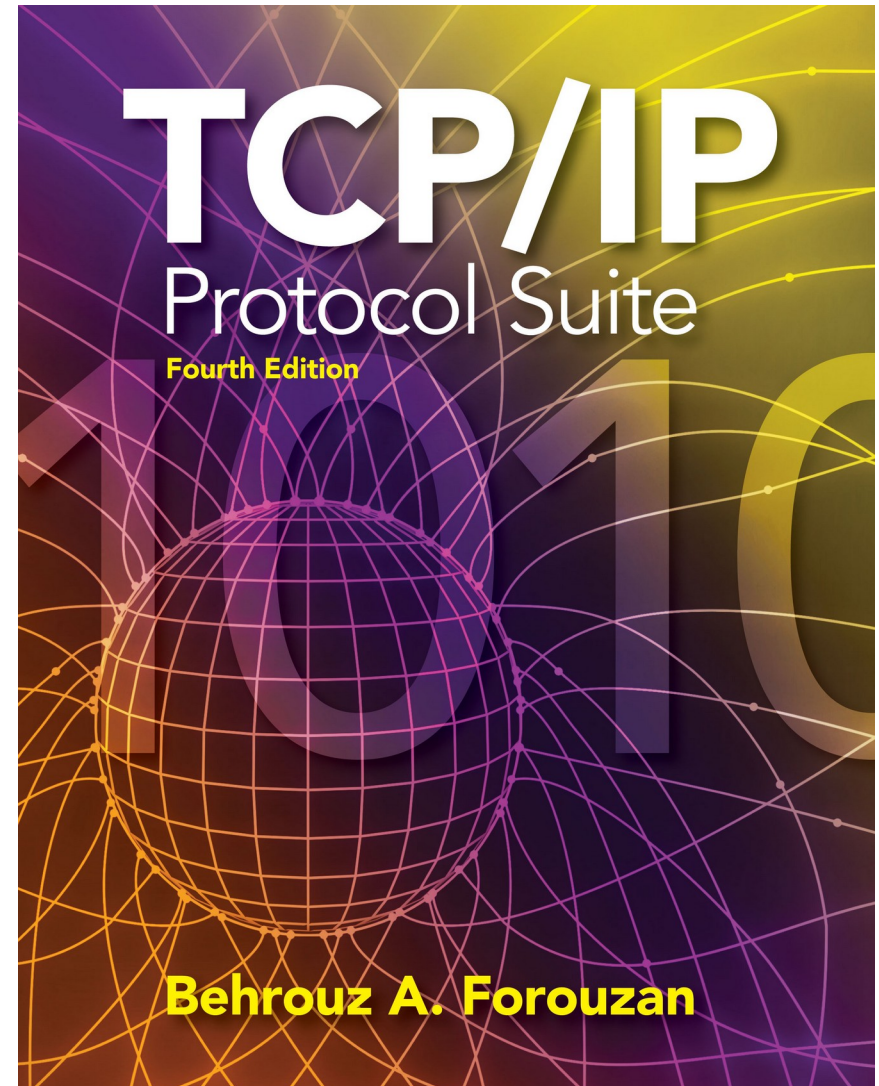


Chapter 13

Introduction to the Transport Layer



OBJECTIVES:

- ❑ To define process-to-process communication at the transport layer and compare it with host-to-host communication at the network layer.
- ❑ To discuss the addressing mechanism at the transport layer, to discuss port numbers, and to define the range of port numbers used for different purposes.
- ❑ To explain the packetizing issue at the transport layer: encapsulation and decapsulation of messages.
- ❑ To discuss multiplexing (many-to-one) and demultiplexing (one-to-many) services provided by the transport layer.
- ❑ To discuss flow control and how it can be achieved at the transport layer.

OBJECTIVES *(continued)*:

- ❑ To discuss error control and how it can be achieved at the transport layer.
- ❑ To discuss congestion control and how it can be achieved at the transport layer.
- ❑ To discuss the connectionless and connection-oriented services at the transport layer and show their implementation using an FSM.

Chapter 13.1 Transport-Layer Services

Outline

13.2 Transport-Layer Protocols

13-1 TRANSPORT-LAYER SERVICES

As we discussed in Chapter 2, the transport layer is located between the network layer and the application layer. The transport layer is responsible for providing services to the application layer; it receives services from the network layer. In this section, we discuss the services that can be provided by a transport layer; in the next section, we discuss the principle beyond several transport layer protocols.

Topics Discussed in the Section

- ✓ **Process-to-Process Communication**
- ✓ **Addressing: Port Numbers**
- ✓ **Encapsulation and Decapsulation**
- ✓ **Multiplexing and Demultiplexing**
- ✓ **Flow Control**
- ✓ **Error Control**
- ✓ **Congestion Control**
- ✓ **Connectionless and Connection-Oriented Services**

Figure 13.1 *Network layer versus transport layer*

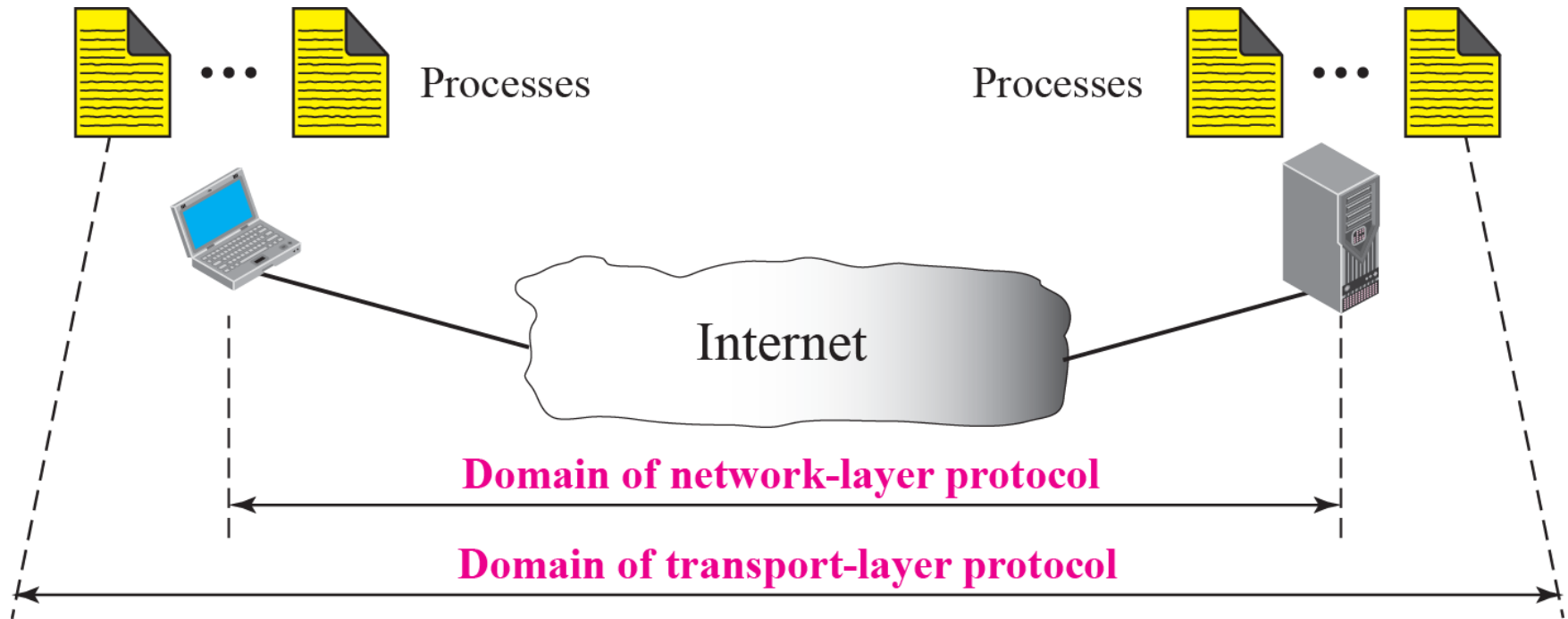


Figure 13.2 *Port numbers*

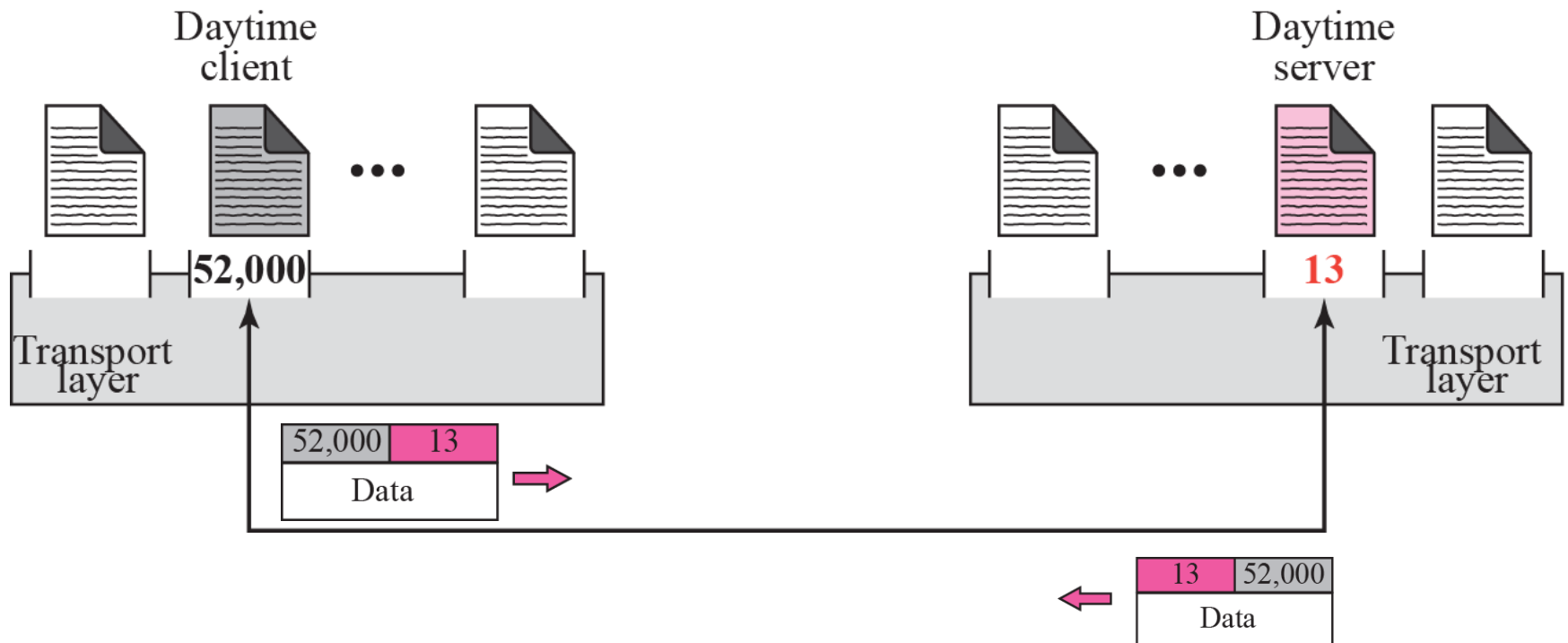


Figure 13.3 *IP addresses versus port numbers*

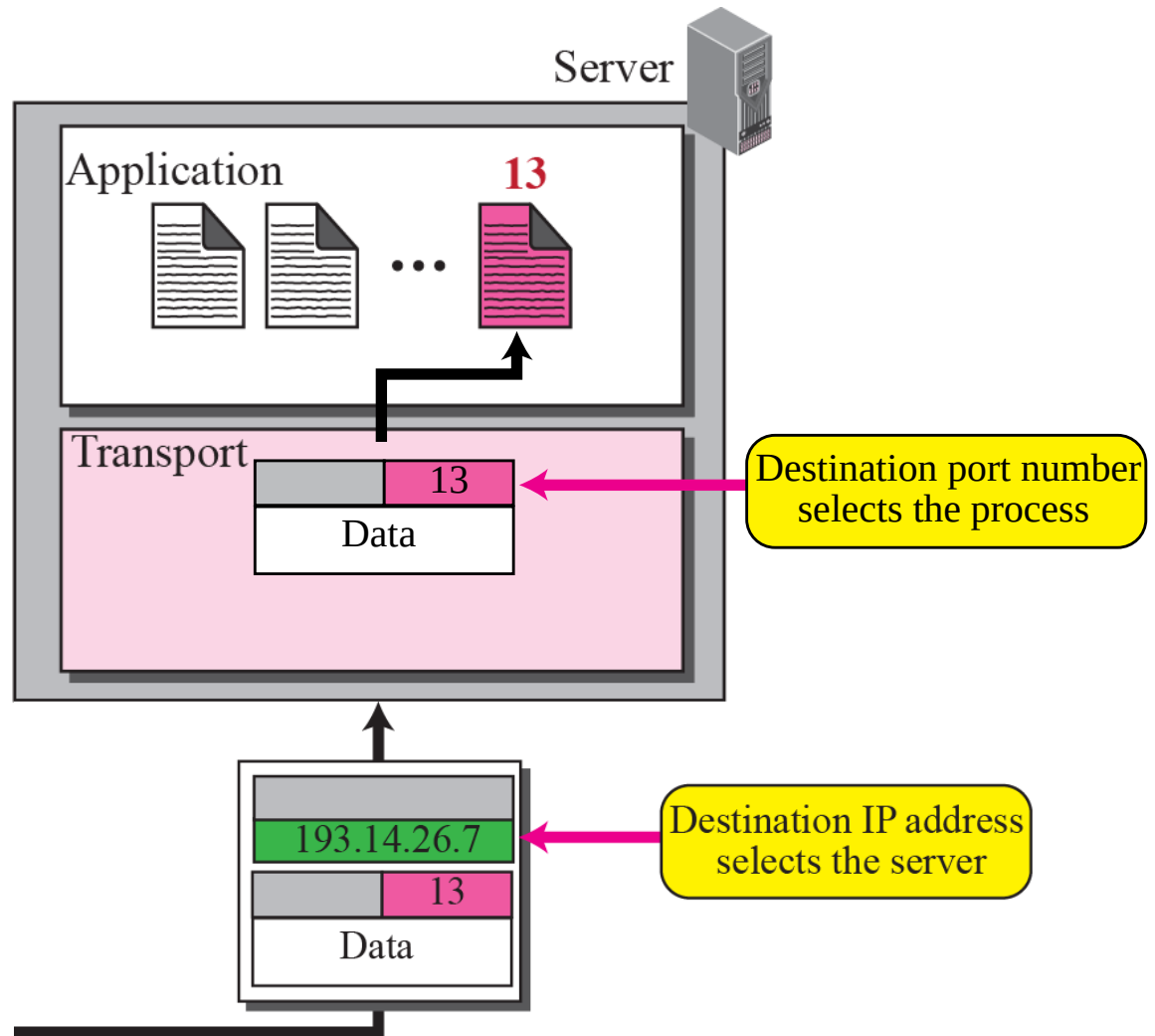
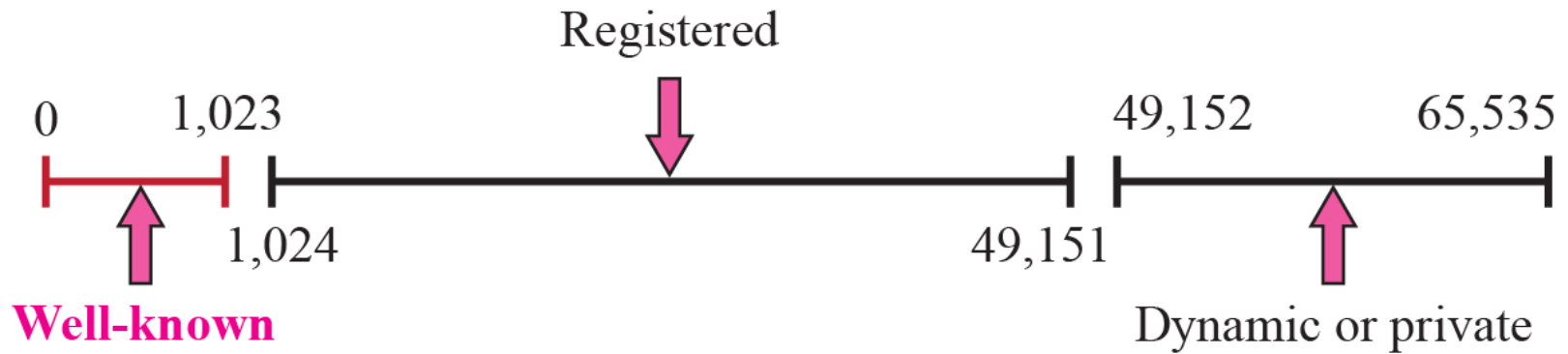


Figure 13.4 *ICANN ranges*



Note

The well-known port numbers are less than 1,024.

Example 13.1

In UNIX, the well-known ports are stored in a file called `/etc/services`. Each line in this file gives the name of the server and the well-known port number. We can use the `grep` utility to extract the line corresponding to the desired application. The following shows the port for TFTP. Note that TFTP can use port 69 on either UDP or TCP. SNMP (see Chapter 24) uses two port numbers (161 and

```
$grep tftp /etc/services
tftp 69/tcp
tftp 69/udp
```

```
$grep snmp /etc/services
snmp161/tcp#Simple Net Mgmt Proto
snmp161/udp#Simple Net Mgmt Proto
snmptrap162/udp#Traps for SNMP
```

Figure 13.5 *Socket address*

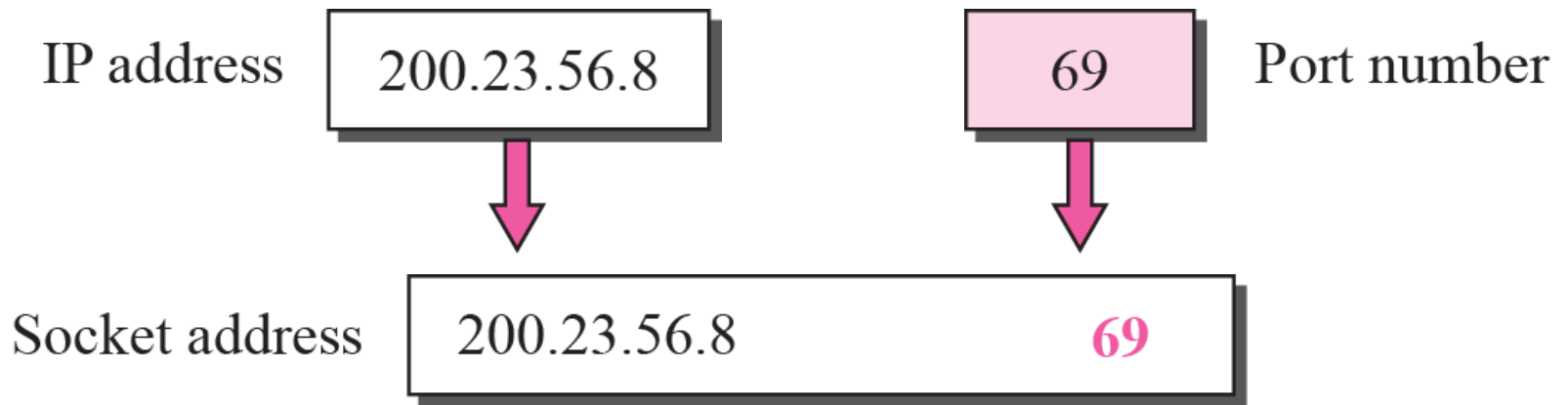


Figure 13.6 *Encapsulation and decapsulation*

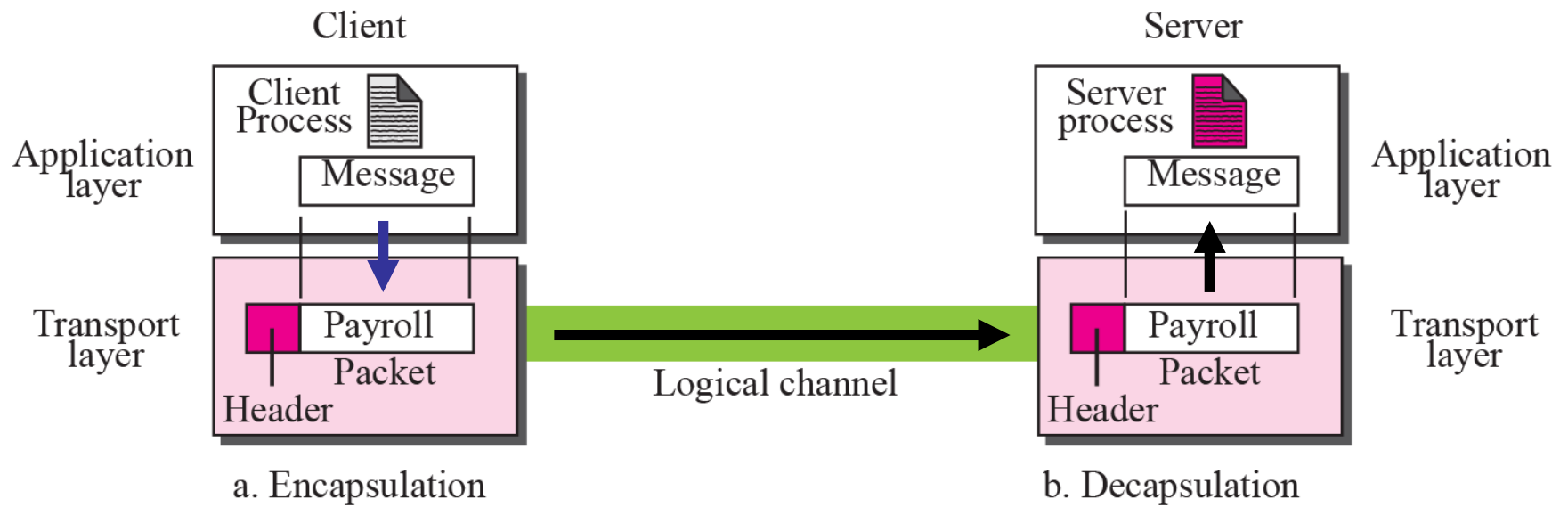


Figure 13.7 *Multiplexing and demultiplexing*

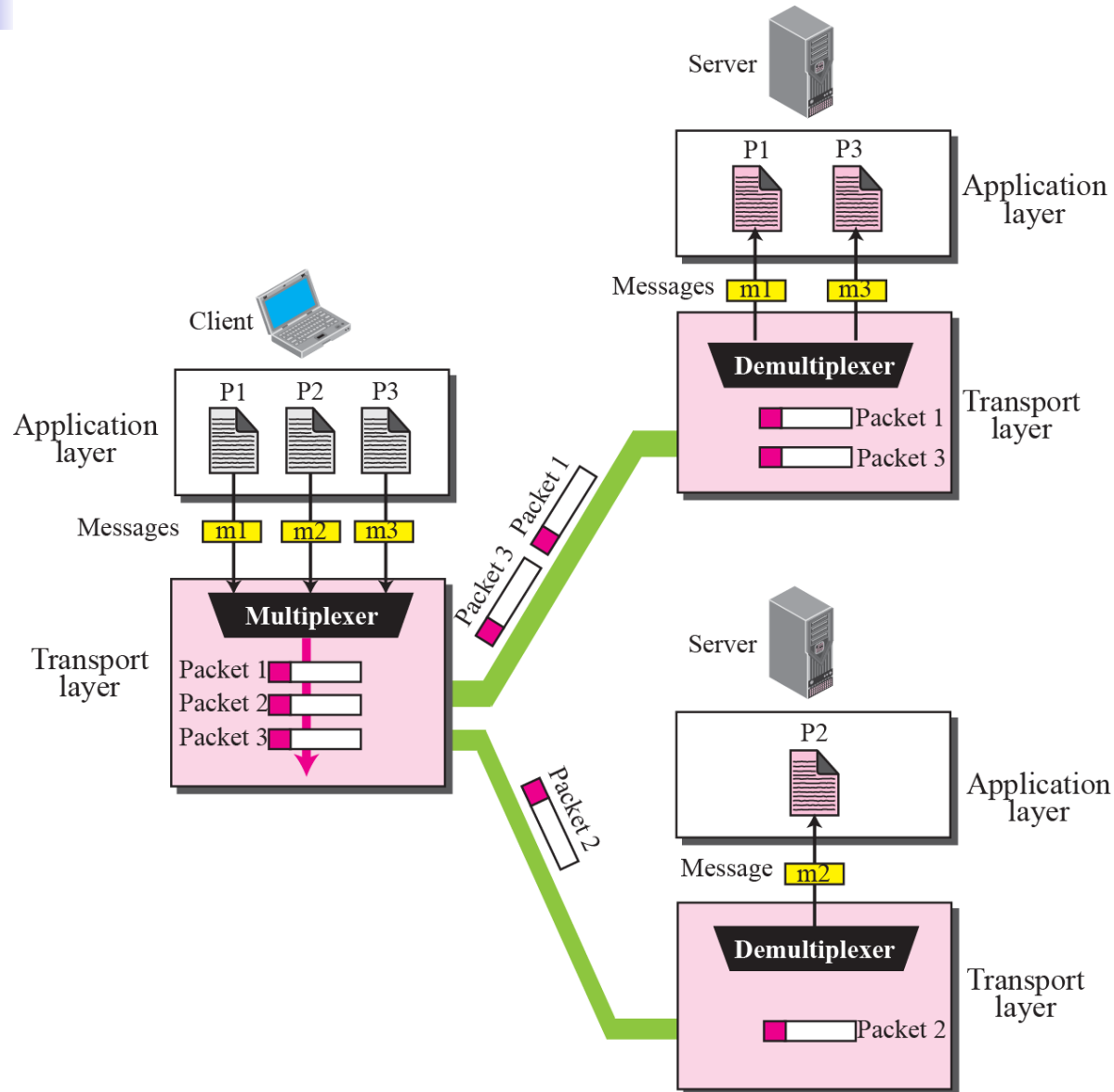
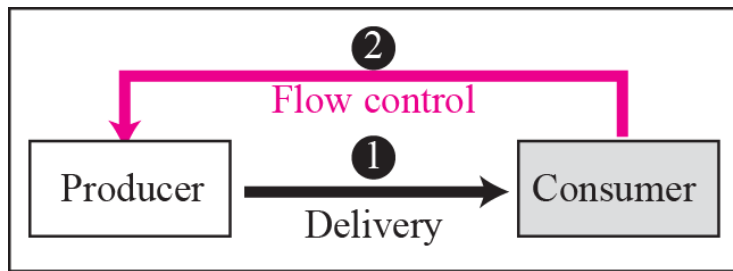
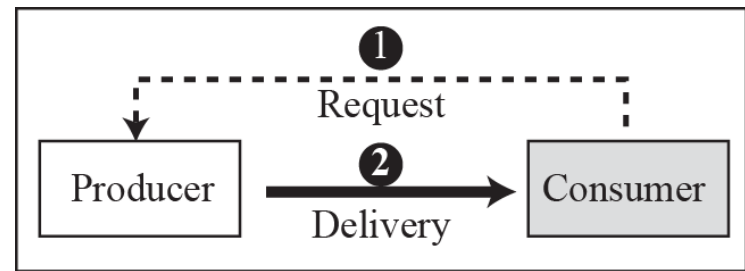


Figure 13.8 *Pushing or pulling*

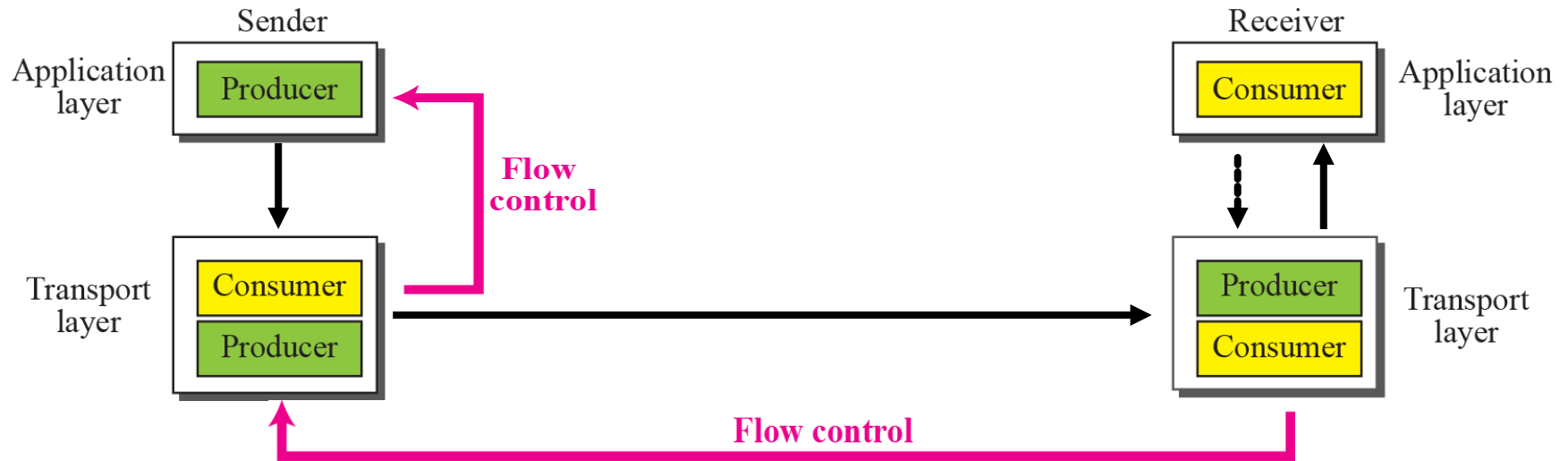


a. Pushing



b. Pulling

Figure 13.9 *Flow control at the transport layer*

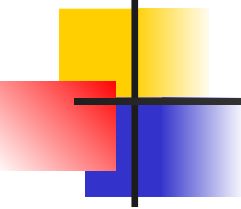


Example 13.2

The above discussion requires that the consumers communicate with the producers in two occasions: when the buffer is full and when there are vacancies. If the two parties use a buffer of only one slot, the communication can be easier. Assume that each transport layer uses one single memory location to hold a packet. When this single slot in the sending transport layer is empty, the sending transport layer sends a note to the application layer to send its next chunk; when this single slot in the receiving transport layer is empty, it sends an acknowledgment to the sending transport layer to send its next packet. As we will see later, this type of flow control, using a single-slot buffer at the sender and the receiver, is inefficient.

Figure 13.10 *Error control at the transport layer*





Note

For error control, the sequence numbers are modulo 2^m , where m is the size of the sequence number field in bits.

Figure 13.11 *Sliding window in circular format*

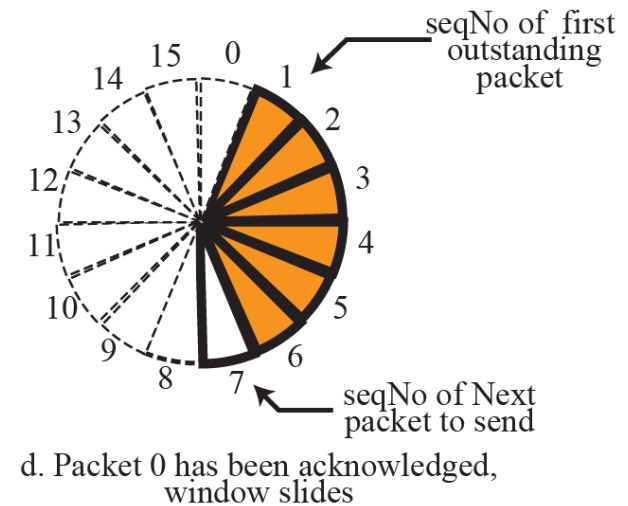
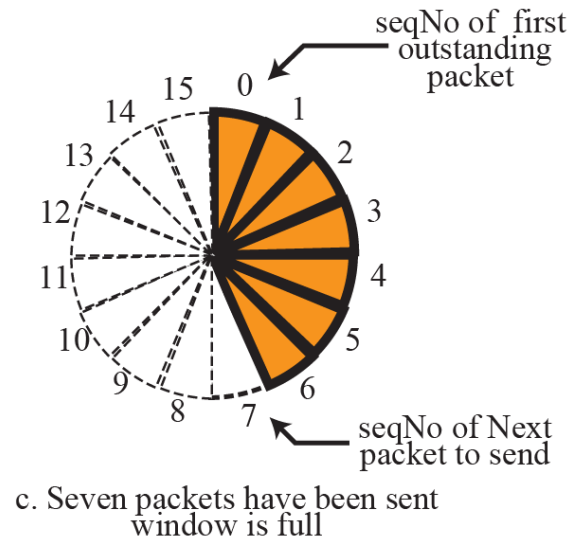
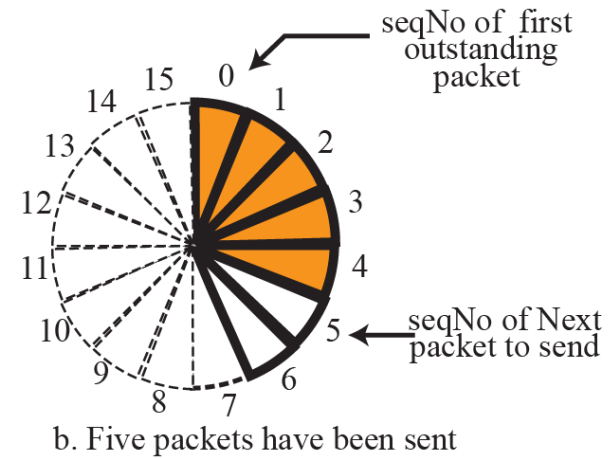
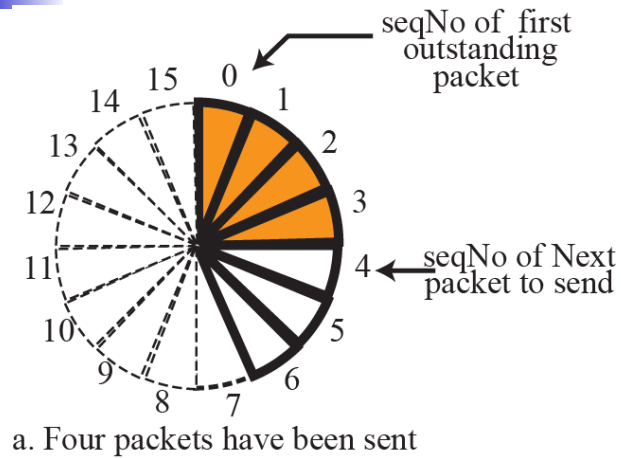


Figure 13.12 *Sliding window in linear format*



a. Four packets have been sent



b. Five packets have been sent



c. Seven packets have been sent
window is full



d. Packet 0 have been acknowledged
and window slid

Figure 13.13 *Connectionless service*

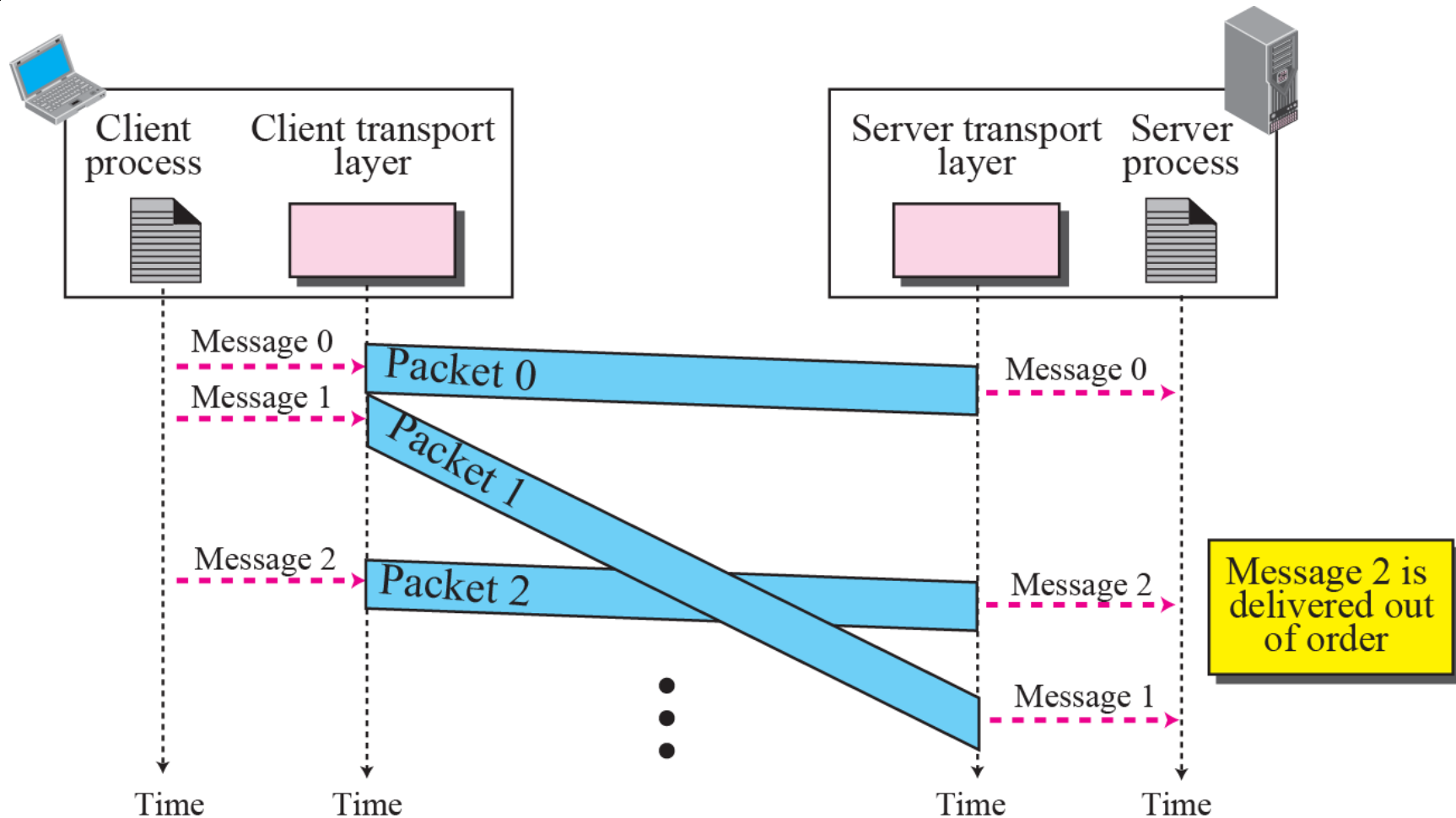


Figure 13.14 *Connection-oriented service*

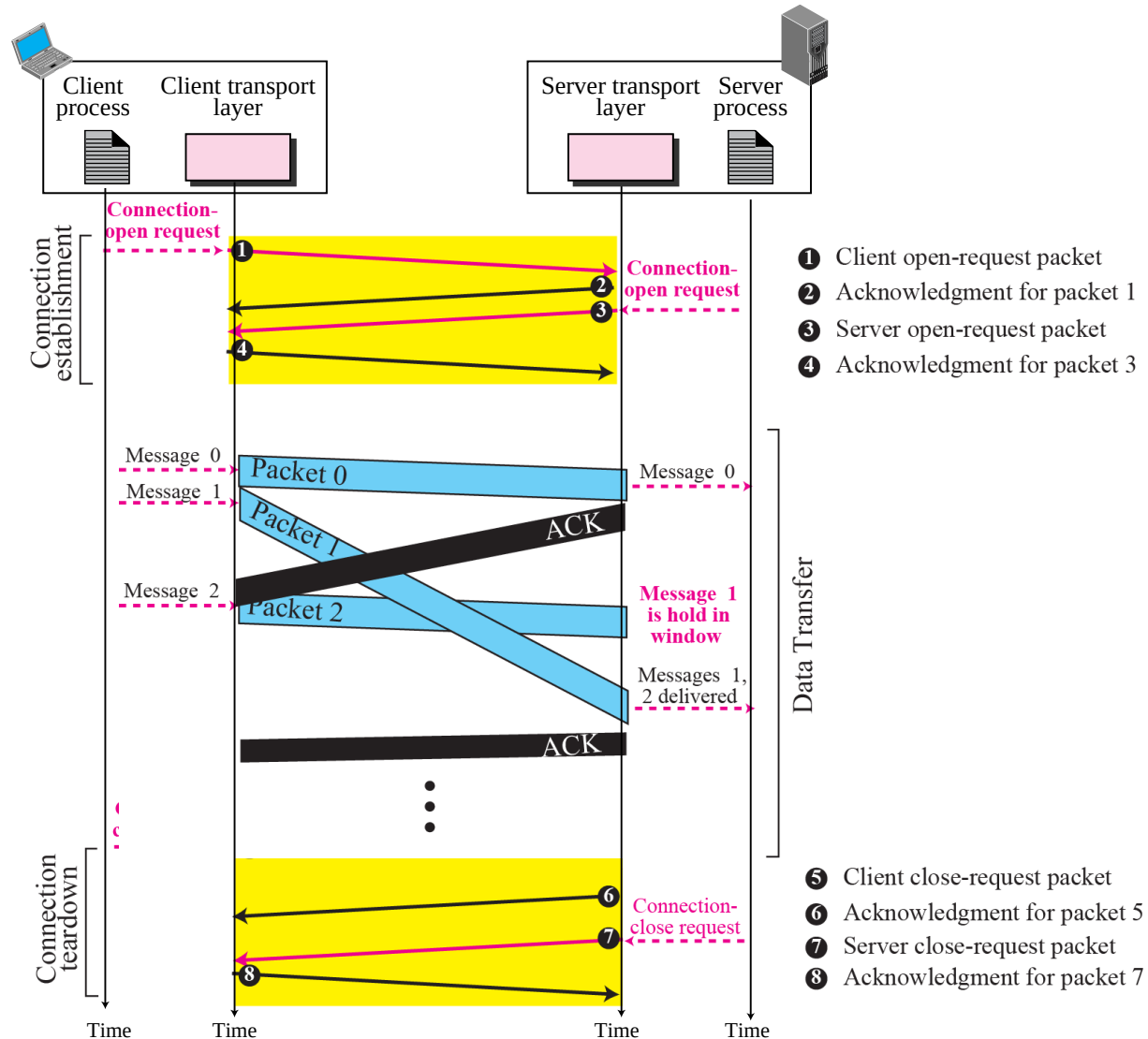
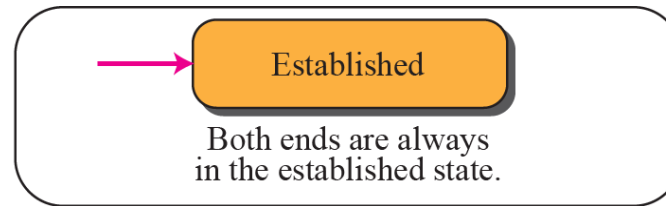


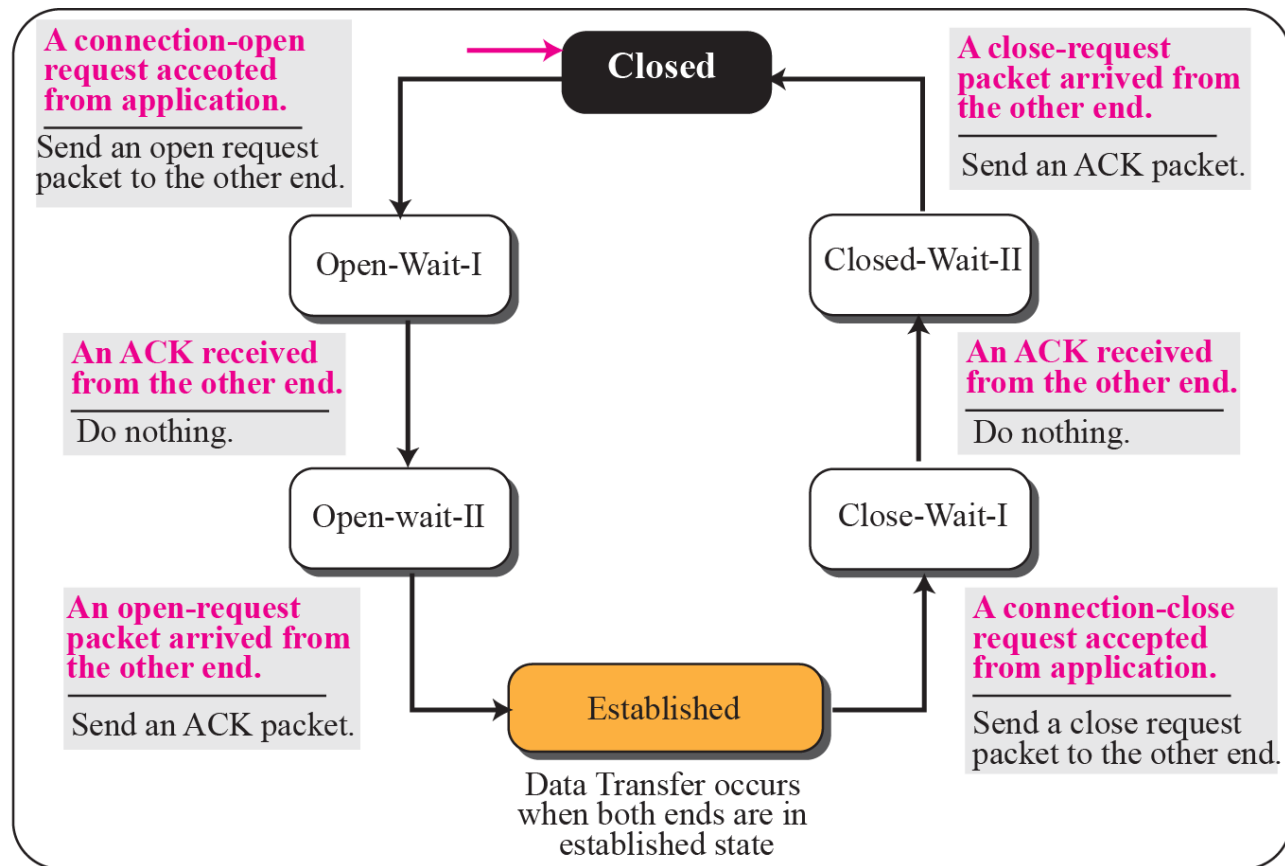
Figure 13.15 *Connectionless and connection-oriented services as FSMs*

Note:
The colored
arrow shows the
starting state.

FSM for
connectionless
transport layer



FSM for
connection-oriented
transport layer



13-2 TRANSPORT-LAYER PROTOCOLS

We can create a transport-layer protocol by combining a set of services described in the previous sections. To better understand the behavior of these protocols, we start with the simplest one and gradually add more complexity. The TCP/IP protocol uses a transport layer protocol that is either a modification or a combination of some of these protocols.

Topics Discussed in the Section

- ✓ **Simple Protocol**
- ✓ **Stop-and-Wait Protocol**
- ✓ **Go-Back- N Protocol**
- ✓ **Selective-Repeat Protocol**
- ✓ **Bidirectional Protocols: Piggybacking**